

文本统计分析

题目描述

有一个文件，包含以一定规则写作的文本，请统计文件中包含的文本数量。

规则如下：

1. 文本以 ";" 分隔，最后一条可以没有 ";"，但空文本不能算语句，比如

```
COMMAND A;
```

只能算一条语句。

注意，无字符/空白字符/制表符都算作"空"文本；

2. 文本可以跨行，比如下面，是一条文本，而不是三条

```
COMMAND A
AND
COMMAND B;
```

3. 文本支持字符串，字符串为成对的单引号(')或者成对的双引号(")，字符串可能出现用转义字符(\)处理的单双引号("your input is'")和转义字符本身，比如

```
COMMAND A "Say \"hello\"";
```

4. 支持注释，可以出现在字符串之外的任意位置注释以"--"开头，到换行结束，比如

```
COMMAND A; -- this is comment
COMMAND -- comment
A AND COMMAND B;
```



注意字符串内的"--"，不是注释。

输入描述

文本文件

输出描述

包含的文本数量

用例

输入	<pre>COMMAND TABLE IF EXISTS "UNITED STATE"; COMMAND A GREAT (ID ADSAB, download_length INTE-GER, -- test file_name TEXT, guid TEXT, mime_type TEXT, notifica-tionid INTEGER, original_file_name TEXT, pause_reason_type INTEGER, resumable_flag INTEGER, start_time INTEGER, state INTEGER, folder TEXT, path TEXT, total_length INTE-GER, url TEXT);</pre>
输出	2

说明	无
----	---

题目解析

梳理一下题目信息：

- 一行中可以有多条文本，比如一行输入为： `abc;abc;abc`；此时该行有三条文本。
- 一条文本以分号结束
- 成对双引号、成对单引号中的分号，无法作为一条文本结束的标志
- 注释符号后面的分号，无法作为一条文本结束的标志
- 成对双引号、成对单引号中的注释符号失效
- 成对双引号、成对单引号中可能存在转义字符

本题要我们求解文本条数，其实可以转化为求解有效分号个数。

而无效分号，其实就是：

- 成对双引号、成对单引号中的分号
- 注释符号后面的分号

我的解题思路如下，假设一行字符串为line：

1. 首先，将line中的转义双引号、转义单引号去掉，避免影响成对双引号、成对单引号的判断
2. 然后，将line中的成对双引号及其中内容、成对单引号及其中内容去除，这样可以简单粗暴的去除成对双引号、成对单引号中的注释符号、分号
3. 之后，将line中注释内容（即：‘`#`’及其后内容）去掉，这样可以简单粗暴的去除注释中的分号
4. 此时line中就只会剩下分号，我们需要处理一些特殊情况，比如空文本不能算语句，我们可以将line中的空白字符去除掉
5. 经过上一步处理，如果line中存在空白文本，则去除空白字符后，必然会出现类似于 `abc;;` 这种连续分号的现象，此时我们应该将连续分号替换为单个分号
6. 经过上一步处理，还有一种特殊情况，比如 `;abc` 这种，我们需要去除开头分号

7. 最后，line中剩余几个分号，就代表此行存在几个文本

上面逻辑，如果使用 **字符串操作** 来写，则较为复杂，我这里直接使用了正则表达式来处理。

另外，需要注意的是，题目描述中说

文本以“;”分隔，最后一条可以没有“;”

这句话的意思应该是，最后一行输入中的最后一条文本，可以没有分号。

比如最后一行为 **abc;abc**

此时最后一行其实有两条文本，因为最后一条可以没有分号。

关于最后一行，我们可以粗暴的直接给它拼接一个分号，这样话，无论最后一行的最后一条有没有分号，我们都能保证前面的逻辑可以统计到最后一行。

2023.09.05 根据考友提示，上面逻辑存在漏洞，成对双引号、成对单引号之间的内容不能直接替换为空串，因为对于如下情况：

`"abc";"abc";"abc";`

其实应该是3行。

我们可以将成对双引号、成对单引号之间的内容替换为简单串，比如a

2024.02.08

之前题目描述中关于注释的描述好像抄错了。

- 实际考试题目中，是以 "--" 两个-作为注释标志
- 之前博客是将 "-" 一个-作为注释表示

另外补充一个用例，比如：

```
\";
```

其中\"也是一个字符，算是有内容，因此这个用例有1个文本。

之前代码直接将\"和\"替换为了空，因此之前代码逻辑输入上面用例会返回0个文本

JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6    const lines = [];
7
8    let line;
9    while (line = await readline()) {
10      lines.push(line);
11    }
12
13    console.log(getResult(lines));
14  })();
15
16  function getResult(lines) {
17    let s = lines
18      .map(
19        (line) =>
20          line
21            .replaceAll(/\\\[\"']/g, "a") // 替换\"和\"为普通字符
22            .replaceAll(/\".*?\"/g, "a") // 将成对双引号及其中内容替换为普通串
23            .replaceAll(/'.*?'/g, "a") // 将成对单引号及其中内容替换为普通串
24            .replaceAll(/--.+/g, "") // 将--及其往后的注释替换为空串
```

运行

```

25 |         .replaceAll(/\s+/g, "") // 将空白字符替换为空串
26 |
27 |     )
28 |     .join("");
29 |
30 | // 题目描述说：文本以“;”分隔，最后一条可以没有“;”
31 | // 为了避免复杂处理，这里无论最后一条文本有没有分号，都加一个
32 | s += ";";
33 |
34 | // 下面处理主要是为了处理跨行文本
35 | /**
36 |  * 比如
37 |  * aaaa;
38 |  * ;aaaa
39 |  *
40 |  * 比如
41 |  * ;aaaa
42 |  * ;aaaa
43 |  */
44 | s = s.replaceAll(/;+/g, ";").replaceAll(/^;/g, "");
45 |
46 | // 记录文本条数
47 | let count = 0;
48 |
49 | for (let c of s) {
50 |     if (c == ";") count++; // 此时一行有几个分号，就代表几条文本
51 | }
52 |
53 | return count;
54 | }

```

```

.replaceAll(/;+/g, ";") // 将连续分号替换为单个分号

```

Java算法源码

```

1 | import java.util.LinkedList;

```

```

2 | import java.util.Scanner;
3 |
4 | public class Main {
5 |     // 输入获取
6 |     public static void main(String[] args) {
7 |         Scanner sc = new Scanner(System.in);
8 |
9 |         LinkedList<String> lines = new LinkedList<>();
10 |        while (sc.hasNextLine()) {
11 |            String line = sc.nextLine();
12 |
13 |            // 本地测试时，以输入空行作为结束条件，实际考试时无需此逻辑
14 |            //      if (line.length() == 0) break;
15 |
16 |            lines.add(line);
17 |        }
18 |
19 |        System.out.println(getResult(lines));
20 |    }
21 |
22 |    // 算法入口
23 |    public static int getResult(LinkedList<String> lines) {
24 |        StringBuilder sb = new StringBuilder();
25 |
26 |        for (String line : lines) {
27 |            line =
28 |                line.replaceAll("\\\\\\"["'"]", "a") // 替换\"和\'为普通字符
29 |                .replaceAll("\\\".*?\\\"", "a") // 将成对双引号及其中内容替换为普通串
30 |                .replaceAll("'.*?'", "a") // 将成对单引号及其中内容替换为普通串
31 |                .replaceAll("--.+ ", "") // 将--及其往后的注释替换为空串
32 |                .replaceAll("\\s+", "") // 将空白字符替换为空串
33 |                .replaceAll(";+", ";"); // 将连续分号替换为单个分号
34 |
35 |            sb.append(line);
36 |        }
37 |
38 |        // 题目描述说：文本以“;”分隔，最后一条可以没有“;”

```

```

39 // 为了避免复杂处理，这里无论最后一条文本有没有分号，都加一个
40 sb.append(";");
41
42 // 下面处理主要是为了处理跨行文本
43 /**
44  * 比如
45  * aaaa;
46  * ;aaaa
47  *
48  * 比如
49  * ;aaaa
50  * ;aaaa
51  */
52 String s = sb.toString().replaceAll(";", ";").replaceAll("^;", "");
53
54 // 记录文本条数
55 int count = 0;
56
57 for (int i = 0; i < s.length(); i++) {
58     if (s.charAt(i) == ';') count++;
59 }
60
61 return count;
62 }
63 }

```

Python算法源码

```

1 import re
2 import sys
3
4
5 def fn(s):
6     s = re.sub("\\\\[\\'"]", "a", s) # 替换\"和\'为普通字符
7     s = re.sub("\\\".*?\\\"", "a", s) # 将成对双引号及其中内容替换为普通串

```



```

8      s = re.sub("'.*?'", "a", s) # 将成对单引号及其中内容替换为普通串 9
10     s = re.sub("\\s+", "", s) # 将空白字符替换为空串
11     s = re.sub(";+", ";", s) # 将连续分号替换为单个分号
12     return s
13
14
15 # 算法入口
16 def getResult(lines):
17     s = "".join(map(fn, lines))
18
19     # 题目描述说：文本以”;”分隔，最后一条可以没有”;”
20     # 为了避免复杂处理，这里无论最后一条文本有没有分号，都加一个
21     s += ";"
22
23     # 下面处理主要是为了处理跨行文本
24     """
25     比如
26     aaaa;
27     ;aaaa
28
29     比如
30     ;aaaa
31     ;aaaa
32     """
33     s = re.sub(";+", ";", s)
34     s = re.sub("^;", "", s)
35
36     # 记录文本条数
37     count = 0
38
39     for c in s:
40         if c == ';':
41             count += 1 # 此时一行有几个分号，就代表几条文本
42
43     return count
44
45

```

```

s = re.sub("--.+", "", s) # 将--及其往后的注释替换为空串

```

```
46 | # 输入获取
    | 47 | lines = []
48 |
49 | # 注意该读取方式只适合Python3编译器，考试时注意选择Python3编译器
50 | for line in sys.stdin:
51 |     lines.append(line)
52 |
53 | print(getResult(lines))
```

满分解法

下面是一个满分解法，但是感觉漏洞还是比较多的，比如下面用例：

```
\";
```

```
\\;
```

```
"-;
```

```
" ";
```

大家可以自己先思考一下上面用例的输出应该是什么，然后看看下面满分解法的输出是什么。

下面解法已加入详细注释，因此不再赘述解析。

Java算法源码

```
1 | import java.util.Scanner;
```

```
2 import java.util.StringJoiner; 3 |
4 public class Main {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         // 将文件内容压缩为一行，换行使用换行符\n代替
9         StringJoiner sj = new StringJoiner("\n");
10        while (sc.hasNextLine()) {
11            String line = sc.nextLine();
12
13            // 本地测试时，以输入空行作为结束条件，实际考试时无需此逻辑
14            //      if (line.length() == 0) break;
15
16            sj.add(line);
17        }
18
19        System.out.println(getResult(sj.toString()));
20    }
21
22    public static int getResult(String s) {
23        // 记录题解，即文件中包含的文本数量
24        int ans = 0;
25
26        // 当前文本的“非空”内容长度
27        int content_length = 0;
28
29        // 单引号开闭状态，当前解法默认单引号必然成对出现
30        boolean sq_flag = false;
31        // 双引号开闭状态，当前解法默认双引号必然成对出现
32        boolean dq_flag = false;
33
34        // 定义一个指针 i 用于扫描文件内容
35        int i = 0;
36
37        while (i < s.length()) {
38            // 当前正在被扫描的字符
39            char c = s.charAt(i);
```

```

40
41 switch (c) {
42     case '\\': // 如果扫描到转义\符号，则自动跳过转义符号及其后面的转义字符
43         i++;
44         break;
45     case '\"': // 如果扫描到双引号
46         if (!sq_flag) { // 如果该双引号不在成对单引号内部
47             dq_flag = !dq_flag; // 则双引号开闭状态取反
48         }
49         break;
50     case '\': // 如果扫描到单引号
51         if (!dq_flag) { // 如果该单引号不在成对双引号内部
52             sq_flag = !sq_flag; // 则单引号开闭状态取反
53         }
54         break;
55     case '-': // 如果扫描到 '-'，则可能是注释符号，也可能不是
56         if (!sq_flag && !dq_flag) { // 如果该 '-' 处于成对双向号，或成对单引号内部，则不是注释符号，否则可能是注释符号
57             if (++i < s.length() && s.charAt(i) == '-') { // 如果该 '-' 后面紧跟着的字符还是 '-'，则确定当前遇到注释符号
58                 while (++i < s.length() && s.charAt(i) != '\n') {} // 注释符号后面的内容（直到行末）都不算文本，即不会增加文本数量
59             }
60         }
61         break;
62     case ';': // 如果扫描到 ';'
63         if (!sq_flag & !dq_flag && content_length > 0) { // 该分号不处于成对单引号或成对双引号内部，且分号前存在非空内容
64             ans++; // 则文本数量++
65             content_length = 0; // 继续统计下一个文本
66         }
67         break;
68     default: // 如果扫描到其他字符，且该字符非空白字符，则非空文本内容长度++
69         if (!Character.isWhitespace(c)) {
70             content_length++;
71         }
72         break;
73 }
74
75 // 继续扫描下一个字符
76 i++;

```

```

77 |     }
78 |
79 | // 题目说最后一个文本可以没有分号，即最后的文本只有存在非空内容，即为一个文本
80 | if (content_length > 0) {
81 |     ans++;
82 | }
83 |
84 | return ans;
85 | }
86 | }

```

JS算法源码

```

1 | const rl = require("readline").createInterface({ input: process.stdin });
2 | var iter = rl[Symbol.asyncIterator]();
3 | const readline = async () => (await iter.next()).value;
4 |
5 | void (async function () {
6 |     const lines = [];
7 |
8 |     let line;
9 |     while ((line = await readline())) {
10 |         lines.push(line);
11 |     }
12 |
13 | // 将文件内容压缩为一行，换行使用换行符\n代替
14 | const s = lines.join("\n");
15 |
16 | console.log(getResult(s));
17 | })();
18 |
19 | function getResult(s) {
20 |     // 记录题解，即文件中包含的文本数量
21 |     let ans = 0;
22 |
23 |     // 当前文本的“非空”内容长度

```

运行

```

24 | let content_length = 0;25 |
26 | // 单引号开闭状态，当前解法默认单引号必然成对出现
27 | let sq_flag = false;
28 | // 双引号开闭状态，当前解法默认双引号必然成对出现
29 | let dq_flag = false;
30 |
31 | // 定义一个指针 i 用于扫描文件内容
32 | let i = 0;
33 |
34 | while (i < s.length) {
35 |     // 当前正在被扫描的字符
36 |     const c = s[i];
37 |
38 |     switch (c) {
39 |         case "\\": // 如果扫描到转义\符号，则自动跳过转义符号及其后面的转义字符
40 |             i++;
41 |             break;
42 |         case '"': // 如果扫描到双引号
43 |             // 如果该双引号不在成对单引号内部
44 |             if (!sq_flag) {
45 |                 dq_flag = !dq_flag; // 则双引号开闭状态取反
46 |             }
47 |             break;
48 |         case "'": // 如果扫描到单引号
49 |             // 如果该单引号不在成对双引号内部
50 |             if (!dq_flag) {
51 |                 sq_flag = !sq_flag; // 则单引号开闭状态取反
52 |             }
53 |             break;
54 |         case "-": // 如果扫描到 '-'，则可能是注释符号，也可能不是
55 |             // 如果该 '-' 处于成对双向号，或成对单引号内部，则不是注释符号，否则可能是注释符号
56 |             if (!sq_flag && !dq_flag) {
57 |                 // 如果该 '-' 后面紧跟着的字符还是 '-'，则确定当前遇到注释符号
58 |                 if (++i < s.length && s[i] == "-") {
59 |                     while (++i < s.length && s[i] != "\n") {} // 注释符号后面的内容（直到行末）都不算文本，即不会增加文本数量
60 |                 }
61 |             }

```

```

62         break;
63     case ";": // 如果扫描到 ';'
64         // 该分号不处于成对单引号或成对双引号内部，且分号前存在非空内容
65         if (!sq_flag && !sq_flag && content_length > 0) {
66             ans++; // 则文本数量++
67             content_length = 0; // 继续统计下一个文本
68         }
69         break;
70     default: // 如果扫描到其他字符，且该字符非空白字符，则非空文本内容长度++
71         if (!/\s/.test(c)) {
72             content_length++;
73         }
74         break;
75     }
76
77     // 继续扫描下一个字符
78     i++;
79 }
80
81 // 题目说最后一个文本可以没有分号，即最后的文本只有存在非空内容，即为一个文本
82 if (content_length > 0) {
83     ans++;
84 }
85
86 return ans;
87 }

```

Python算法源码

```

1 import sys
2
3
4 # 算法入口
5 def getResult(s):
6     # 记录题解，即文件中包含的文本数量

```

```

7 | ans = 0
8 |
9 | # 当前文本的"非空"内容长度
10 | content_length = 0
11 |
12 | # 单引号开闭状态, 当前解法默认单引号必然成对出现
13 | sq_flag = False
14 | # 双引号开闭状态, 当前解法默认双引号必然成对出现
15 | dq_flag = False
16 |
17 | # 定义一个指针 i 用于扫描文件内容
18 | i = 0
19 | while i < len(s):
20 |     # 当前正在被扫描的字符
21 |     c = s[i]
22 |
23 |     if c == '\\': # 如果扫描到转义\符号, 则自动跳过转义符号及其后面的转义字符
24 |         i += 1
25 |     elif c == '\"': # 如果扫描到双引号
26 |         if not sq_flag: # 如果该双引号不在成对单引号内部
27 |             dq_flag = not dq_flag # 则双引号开闭状态取反
28 |     elif c == '\': # 如果扫描到单引号
29 |         if not dq_flag: # 如果该单引号不在成对双引号内部
30 |             sq_flag = not sq_flag # 则单引号开闭状态取反
31 |     elif c == '-': # 如果扫描到 '-', 则可能是注释符号, 也可能不是
32 |         if not sq_flag and not dq_flag: # 如果该 '-' 处于成对双向号, 或成对单引号内部, 则不是注释符号, 否则可能是注释符号
33 |             i += 1
34 |             if i < len(s) and s[i] == '-': # 如果该 '-' 后面紧跟着的字符还是 '-', 则确定当前遇到注释符号
35 |                 i += 1
36 |                 while i < len(s) and s[i] != '\n': # 注释符号后面的内容(直到行末)都不算文本, 即不会增加文本数量
37 |                     i += 1
38 |     elif c == ';': # 如果扫描到 ';'
39 |         if not sq_flag and not dq_flag and content_length > 0: # 该分号不处于成对单引号或成对双引号内部, 且分号前存在非空内容
40 |             ans += 1 # 则文本数量++
41 |             content_length = 0 # 继续统计下一个文本
42 |     else: # 如果扫描到其他字符, 且该字符非空白字符, 则非空文本内容长度++
43 |         if not c.isspace():

```



```

44         content_length += 1
45     # 继续扫描下一个字符
46     i += 1
47
48     # 题目说最后一个文本可以没有分号，即最后的文本只有存在非空内容，即为一个文本
49     if content_length > 0:
50         ans += 1
51
52
53     return ans
54
55
56 # 输入获取
57 def main():
58     lines = []
59
60     # 注意该读取方式只适用于Python3编译器，考试时注意选择Python3编译器
61     for line in sys.stdin:
62         lines.append(line)
63
64     # 将文件内容压缩为一行，换行使用换行符\n代替
65     s = "\n".join(lines)
66     print(getResult(s))
67
68
69 # 算法入口
70 main()

```

C算法源码

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  typedef int bool;
6

```

```
7  #define TRUE 1 8 | #define FALSE 0
9
10 #define MAX_ROW_LENGTH 1000
11 #define MAX_ROW_COUNT 100
12
13 char lines[MAX_ROW_LENGTH * MAX_ROW_COUNT] = {'\0'};
14 char line[MAX_ROW_LENGTH] = {'\0'};
15
16 int solution() {
17     // 记录题解，即文件中包含的文本数量
18     int ans = 0;
19
20     // 当前文本的“非空”内容长度
21     int content_length = 0;
22
23     // 单引号开闭状态，当前解法默认单引号必然成对出现
24     bool sq_flag = FALSE;
25     // 双引号开闭状态，当前解法默认双引号必然成对出现
26     bool dq_flag = FALSE;
27
28     // 定义一个指针 i 用于扫描文件内容
29     int i = 0;
30     unsigned long long size = strlen(lines);
31     while (i < size) {
32         // 当前正在被扫描的字符
33         char c = lines[i];
34
35         if (c == '\\') { // 如果扫描到转义\符号，则自动跳过转义符号及其后面的转义字符
36             i++;
37         } else if (c == '"') { // 如果扫描到双引号
38             if (!sq_flag) { // 如果该双引号不在成对单引号内部
39                 dq_flag = !dq_flag; // 则双引号开闭状态取反
40             }
41         } else if (c == '\'') { // 如果扫描到单引号
42             if (!dq_flag) { // 如果该单引号不在成对双引号内部
43                 sq_flag = !sq_flag; // 则单引号开闭状态取反
44             }
45         }
46     }
47 }
```

```

45 |         } else if (c == '-') { // 如果扫描到 '-', 则可能是注释符号, 也可能不是
46 |
47 |             if (!sq_flag && !dq_flag) { // 如果该 '-' 处于成对双向号, 或成对单引号内部, 则不是注释符号, 否则可能是注释符号
48 |                 if (++i < size && lines[i] == '-') { // 如果该 '-' 后面紧跟着的字符还是 '-', 则确定当前遇到注释符号
49 |                     while (++i < size && lines[i] != '\n') {} // 注释符号后面的内容 (直到行末) 都不算文本, 即不会增加文本数量
50 |                 }
51 |             } else if (c == ';') { // 如果扫描到 ';'
52 |                 if (!sq_flag && !dq_flag && content_length > 0) { // 该分号不处于成对单引号或成对双引号内部, 且分号前存在非空内容
53 |                     ans++; // 则文本数量++
54 |                     content_length = 0; // 继续统计下一个文本
55 |                 }
56 |             } else { // 如果扫描到其他字符, 且该字符非空白字符, 则非空文本内容长度++
57 |                 if (!isspace(c)) {
58 |                     content_length++;
59 |                 }
60 |             }
61 |
62 |             // 继续扫描下一个字符
63 |             i++;
64 |         }
65 |
66 |         // 题目说最后一个文本可以没有分号, 即最后的文本只有存在非空内容, 即为一个文本
67 |         if (content_length > 0) {
68 |             ans++;
69 |         }
70 |
71 |         return ans;
72 |     }
73 |
74 | int main() {
75 |     // 将文件内容压缩为一行, 换行使用换行符\n代替
76 |     while (fgets(line, sizeof(line), stdin) != NULL) {
77 |         strcat(lines, line);
78 |         strcat(lines, "\n");
79 |     }
80 |
81 |     printf("%d\n", solution());

```

```
82 | 83 | return 0;
84 | }
```

C++算法源码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int solution(string &s) {
5      // 记录题解，即文件中包含的文本数量
6      int ans = 0;
7
8      // 当前文本的“非空”内容长度
9      int content_length = 0;
10
11     // 单引号开闭状态，当前解法默认单引号必然成对出现
12     bool sq_flag = false;
13     // 双引号开闭状态，当前解法默认双引号必然成对出现
14     bool dq_flag = false;
15
16     // 定义一个指针 i 用于扫描文件内容
17     int i = 0;
18     while (i < s.length()) {
19         // 当前正在被扫描的字符
20         char c = s[i];
21
22         switch (c) {
23             case '\\': // 如果扫描到转义\符号，则自动跳过转义符号及其后面的转义字符
24                 i++;
25                 break;
26             case '\"': // 如果扫描到双引号
27                 if (!sq_flag) { // 如果该双引号不在成对单引号内部
28                     dq_flag = !dq_flag; // 则双引号开闭状态取反
29                 }
30                 break;
```

```

31     case '\\': // 如果扫描到单引号32 | if (!dq_flag) { // 如果该单引号不在成对双引号内部
32         sq_flag = !sq_flag; // 则单引号开闭状态取反
33     }
34     break;
35     case '-': // 如果扫描到 '-', 则可能是注释符号, 也可能不是
36         if (!sq_flag && !dq_flag) { // 如果该 '-' 处于成对双向号, 或成对单引号内部, 则不是注释符号, 否则可能是注释符号
37             if (++i < s.length() && s[i] == '-') { // 如果该 '-' 后面紧跟着的字符还是 '-', 则确定当前遇到注释符号
38                 while (++i < s.length() && s[i] != '\n') {} // 注释符号后面的内容 (直到行末) 都不算文本, 即不会增加文本数量
39             }
40         }
41         break;
42     case ';': // 如果扫描到 ';'
43         if (!sq_flag && !dq_flag && content_length > 0) { // 该分号不处于成对单引号或成对双引号内部, 且分号前存在非空内容
44             ans++; // 则文本数量++
45             content_length = 0; // 继续统计下一个文本
46         }
47         break;
48     default: // 如果扫描到其他字符, 且该字符非空白字符, 则非空文本内容长度++
49         if (!isspace(c)) {
50             content_length++;
51         }
52         break;
53     }
54
55     // 继续扫描下一个字符
56     i++;
57 }
58
59 // 题目说最后一个文本可以没有分号, 即最后的文本只有存在非空内容, 即为一个文本
60 if (content_length > 0) {
61     ans++;
62 }
63
64 return ans;
65 }
66
67 int main() {

```

```
69 | // 将文件内容压缩为一行，换行使用换行符\n代替
    |
70 | string lines;
71 |
72 | while (!cin.eof()) {
73 |     string line;
74 |     getline(cin, line);
75 |
76 |     lines += line + '\n';
77 | }
78 |
79 | cout << solution(lines) << endl;
80 |
81 | return 0;
82 | }
```